Reviewer Pack — Minimal Tropical Cycle Rank Bounds Randomized Query Complexi…

Entry 20842460a41c · INCONCLUSIVE

SEC autonomous research engine — attribution: Ludovico Kubler

2026-05-29 21:34:00 UTC

Minimal Tropical Cycle Rank Bounds Randomized Query Complexity for XOR

Entry ID: `20842460a41c` **Verdict**: `INCONCLUSIVE` **Recorded**: 2026-05-29 21:34:00 UTC

1. Conjecture

Field A (mathematical branch): Tropical Geometry **Field B** (complexity object): Boolean Function Complexity

Statement:

> For every Boolean function f with an n-bit input, the randomized query complexity Q(f) for XOR is upper-bounded by O(TCR(n)), where TCR(n) is the minimal tropical cycle rank of any representation of f in the max-plus semiring.

Rationale (proposer's reasoning):

> Tropical geometry provides a framework to study discrete functions and their representations. The minimal tropical cycle rank could reveal hidden structures that are not apparent from traditional circuit complexity measures, potentially leading to new insights into the complexities of Boolean functions.

Taxonomy category: `TROPICAL_GEOMETRY` (status at proposal time:)

2. Pre-registration (Popper-style)

Hash (SHA-256 prefix): `a33f83f7f7299dd8`

This hash commits to the conjecture statement, acceptance criterion, and seed list **before** the empirical test runs. Tampering with the test or its data after this hash was computed would be detectable.

Acceptance criterion:

> A result supports the conjecture if the computed Spearman rank correlation coefficient (ρ) is ≥ 0.8 and the mean difference between Q(f) and TCR(n) across all seeds is ≤ 3.

3. Barrier filter (F1)

Final: `PASS`

Barrier	Verdict	Confidence	LLM_1	LLM_2
RELATIVIZATION	UNCERTAIN	1.00	HITS	UNCERTAIN

Barrier	Verdict	Confidence	LLM₁	LLM₂
NATURAL_PROOFS	SAFE	0.50	UNCERTAIN	UNCERTAIN
ALGEBRIZATION	SAFE	0.50	UNCERTAIN	UNCERTAIN
KARP_LIPTON	SAFE	1.00	SAFE	UNCERTAIN

4. Novelty audit

Verdict: `ADJACENT_OK` against 3 hits across arXiv + Semantic Scholar + ECCC

Search queries (3): - `"tropical geometry" AND "Boolean function complexity" AND "randomized query complexity"` - `"max-plus semiring" AND "tropical cycle rank" AND "query complexity for XOR"` - `"minimal tropical cycle rank" IN title AND "Boolean function" IN title AND "randomized query complexity"`

Top relevant hits considered: - [http://arxiv.org/abs/1812.02166v3] On unbalanced Boolean functions with best correlation immunity - [http://arxiv.org/abs/1401.2248v4] Boolean Functions, Quantum Gates, Hamilton Operators, Spin Systems and Computer Algebra - [http://arxiv.org/abs/1004.1724v3] A Class of lattices and boolean functions related to a Manickam-Miklös-Singhi Conjecture

5. Empirical test harness

Multi-seed protocol: 5 seeds (11, 23, 37, 53, 71)
Sandbox: isolated subprocess, stdlib-only, ≤90s wall time per cycle **Execution**: rc=1, elapsed=0.4s

5.1 Generated Python source

```python
import random
import math

def run_trial(seed: int) -> dict:
    random.seed(seed)

    # Generate an n-bit XOR boolean function randomly for each instance.
    n = 10  # Fixed size for simplicity; can be adjusted as needed
    f = [random.choice([0, 1]) for _ in range(2**n)]

    # Compute the minimal tropical cycle rank TCR(n) for each representation of the boolean functi
    def min_tropical_cycle_rank(f):
        m = len(f)
        A = [[math.inf] * m for _ in range(m)]
        for i in range(m):
            for j in range(i, m):
                if f[i] == f[j]:
                    A[i][j] = 0
                    A[j][i] = 0
                else:
                    A[i][j] = 1
                    A[j][i] = 1

        def gaussian_elimination(M):
            rows, cols = len(M), len(M[0])
            for i in range(rows):
                max_row = i
                for j in range(i + 1, rows):
                    if M[j][i] > M[max_row][i]:
```

```
                    max_row = j
            M[i], M[max_row] = M[max_row], M[i]

            factor = M[i][i]
            for j in range(cols):
                M[i][j] /= factor

            for j in range(rows):
                if i != j:
                    factor = M[j][i]
                    for k in range(cols):
                        M[j][k] -= factor * M[i][k]

        return M

    gaussian_elimination(A)

    rank = 0
    for row in A:
        if any(row):
            rank += 1
    return rank

TCR_n = min_tropical_cycle_rank(f)

# Calculate the randomized query complexity Q(f) for XOR.
def xor_query_complexity(n, f):
    queries = []
    for i in range(2**n):
        x = [int(b) for b in format(i, f'0{n}b')]
        y = 0
        for j in range(n):
            if x[j] == 1:
                y ^= f[j]
        queries.append(y)
    return len(set(queries))

Q_f = xor_query_complexity(n, f)

# Calculate the Spearman rank correlation coefficient between Q(f) and TCR(n).
def spearman_rank_correlation(Q_f, TCR_n):
    if not Q_f or not TCR_n:
        return 0
    n = len(Q_f)
    ranks_Q = {x: i for i, x in enumerate(sorted(set(Q_f)), start=1)}
    ranks_T = {x: i for i, x in enumerate(sorted(set(TCR_n)), start=1)}
    rank_diffs = [(ranks_Q[x] - ranks_T[x]) ** 2 for x in Q_f]
    return 1 - (6 * sum(rank_diffs)) / (n * (n**2 - 1))

rho = spearman_rank_correlation([Q_f], [TCR_n])

# Calculate the mean difference between Q(f) and TCR(n).
def mean_difference(Q_f, TCR_n):
    return abs(Q_f[0] - TCR_n[0])

diff = mean_difference([Q_f], [TCR_n])
```

```
    return {
        "metric_name": "Spearman Rank Correlation",
        "metric_value": rho,
        "instances_tested": 1,
        "n_max": n,
        "conjecture_holds": rho >= 0.8 and diff <= 3,
        "counterexample": "" if rho >= 0.8 and diff <= 3 else "Spearman rank correlation < 0.8 or
    }

if __name__ == "__main__":
    import sys
    seeds = [int(s) for s in sys.argv[1:]] or [2, 3, 5, 7, 11, 13, 17, 19, 23, 29] * 3
    results = []

    for seed in seeds:
        result = run_trial(seed)
        print(f"TRIAL: {result}")
        results.append(result)

    mean_rho = sum(r["metric_value"] for r in results) / len(results)
    std_rho = math.sqrt(sum((r["metric_value"] - mean_rho) ** 2 for r in results) / len(results))
    support_fraction = sum(1 for r in results if r["conjecture_holds"]) / len(results)

    if all(r["conjecture_holds"] for r in results):
        print(f"RESULT: SUPPORTED mean={mean_rho} std={std_rho} support_fraction={support_fractio
    elif any(not r["conjecture_holds"] for r in results) and support_fraction >= 0.8:
        first_failing_seed = next(s for s, r in zip(seeds, results) if not r["conjecture_holds"])
        print(f"RESULT: FALSIFIED counterexample='Spearman rank correlation < 0.8 or mean differen
    else:
        print(f"RESULT: INCONCLUSIVE support_fraction={support_fraction}")
```

6. Per-seed results

(no seed-level data — test crashed before producing TRIAL: lines)

7. Test stdout (last 2KB)

```
--STDERR--
Traceback (most recent call last):
  File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_db836d09.py", line 116, in <module>
    result = run_trial(seed)
             ^^^^^^^^^^^^^^^
  File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_db836d09.py", line 67, in run_trial
    TCR_n = min_tropical_cycle_rank(f)
            ^^^^^^^^^^^^^^^^^^^^^^^^^^
  File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_db836d09.py", line 59, in min_tropical_
    gaussian_elimination(A)
  File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_db836d09.py", line 49, in gaussian_elim
    M[i][j] /= factor
ZeroDivisionError: float division by zero
```

8. Critic adversarial review

Critic verdict: CHALLENGE

Critic reasoning:

skipped: test crashed before producing data

9. Final verdict & safety rail

Verdict: `INCONCLUSIVE`

Reasoning:

The test crashed due to a division by zero error before producing data that could be used to evaluate the conjecture. | next: Investigate and fix the division by zero error in the code, then rerun the test with a different seed or input to verify the conjecture.

11. Audit log (LLM calls)

Total LLM calls: 9

#	Phase	Provider	Model	Tokens in	out	Latency (ms)
1	propose	ollama_remote	glm4:latest	0	0	13839
2	preregistration	ollama_remote	glm4:latest	0	0	10443
3	novelty	ollama_remote	glm4:latest	0	0	8668
4	novelty	ollama_remote	glm4:latest	0	0	9347
5	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	13624
6	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	8638
7	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	44429
8	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	44739
9	judge	ollama_remote	glm4:latest	0	0	16974

Totals: 0 input tokens, 0 output tokens, ~$0.0000 cost, 170701 ms total latency. Provider mix: {'ollama_remote': 9}

(full prompt+response transcripts available in `research/audit/20842460a41c.jsonl`)

12. Reproducibility

To reproduce this cycle:

1. Apply the test harness in section 5.1 with seeds 11,23,37,53,71
2. The Python sandbox is pure-stdlib; any Python 3.11+ should suffice
3. The full LLM transcripts are in `research/audit/20842460a41c.jsonl` for verification of provenance
4. The pre-registration hash in section 2 commits to the test design before execution; tampering would be detectable.

Sandbox file: `research/pvsnp_sandbox/test_*.py` (per-cycle)
Replay tarball: `research/replay/20842460a41c.tar.gz` (if generated)